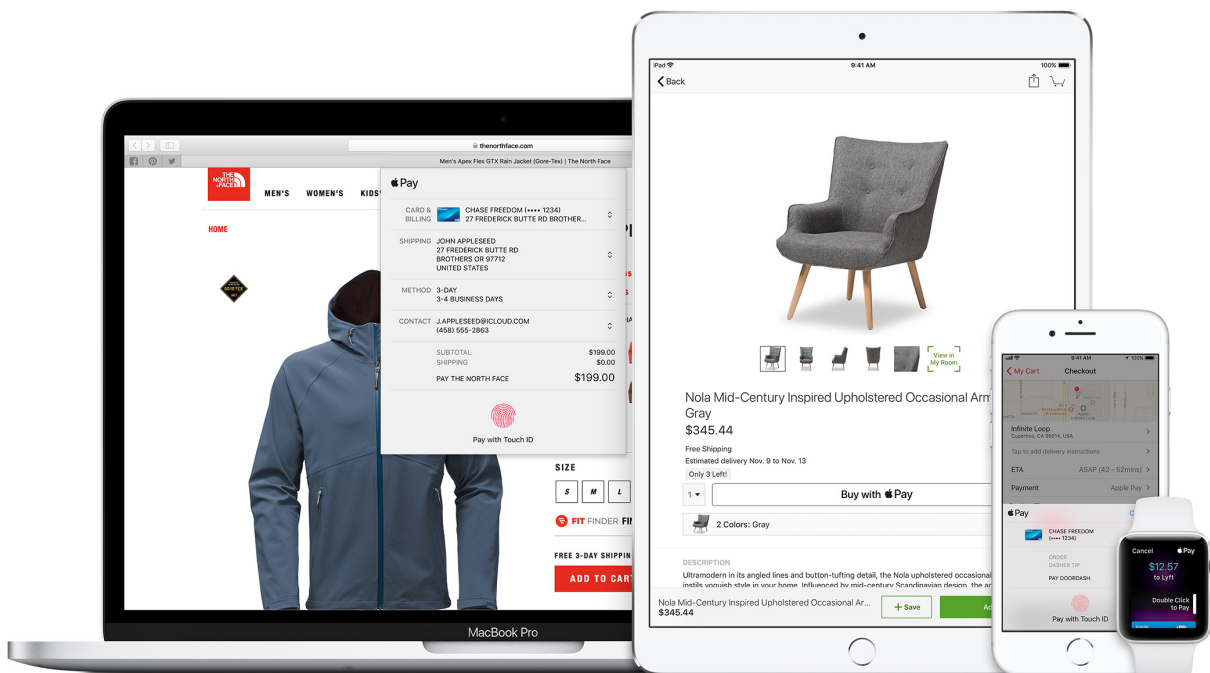




Getting started guide for PSPs



Apple Inc.

Version 1.3

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

© 2018 Apple Inc. All rights reserved. Apple, the Apple logo, Apple Pay, and Wallet are all trademarks of Apple Inc., registered in the U.S. and other countries.

Quick Start	3
Introduction.....	4
Pre-requisites	4
How Apple Pay Works	5
Presenting the Apple Pay Button	5
Presenting the Payment Sheet	5
The Payment Token	5
Merchant On-boarding.....	7
Apple Pay on the Web.....	7
Key Management.....	8
Processing Apple Pay	10
Supported Transaction Types and Business Models	10
Apple Pay Payment Object.....	12
Authorizing the Payment.....	14
Decrypting the Payment Data	14
Constructing the Authorization Message	15
Payment Processor Integration Methods	16
Direct API Integration	16
JavaScript & iOS Code Libraries.....	16
Hosted Payment Forms.....	17
iframes	17
Sandbox Testing	18
Documentation.....	18
Reporting	18
Appendices	19
Appendix A1.....	19
Appendix A2.....	22
Appendix A3	24

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

Quick Start

Key Actions for Supporting Apple Pay

- 1. Familiarise yourself with how Apple Pay works** - see [here](#) for background information on how Apple Pay works
- 2. Extend your integration points to support Apple Pay** - see [here](#) for advice on integrations types
- 3. Manage decryption of the Apple Pay data for merchants** see [here](#) for advice on decryption
- 4. Simplify on-boarding for merchants** - see [here](#) for on-boarding advice
- 5. Enable merchants to report on Apple Pay transactions**

Introduction

Apple Pay provides an easy and secure way to pay in iOS apps, watchOS apps, and websites on Safari. And now, nonprofits can accept donations within their apps and websites with Apple Pay.

By using Face ID, Touch ID or double-clicking Apple Watch, users can quickly and securely provide their payment, shipping, and contact information to check out. Customers love the simplicity of Apple Pay, and merchants love the increased conversion rates and new user adoption that come with it.

Further details about getting started with Apple Pay can be found [here](#).

Pre-requisites

1. Payment processor operates in the [supported regions](#)
2. Payment processor must support 3D Secure. Specifically, payment processor must be capable of receiving the appropriate fields below for the current card networks they support:

	3D Secure Program Name	Cryptogram	Authentication Result
Visa	Verified by Visa	CAVV	ECI
MasterCard	SecureCode	AAV (via UCAF field)	ECI
American Express	SafeKey	AEVV	ECI
Discover	ProtectBuy	CAVV	ECI
UnionPay	UnionPay Online Payments (UPOP)	CAVV	ECI
JCB	J/Secure	CAVV	ECI

Note: Visa Token Services (VTS) certification and/or MasterCard Digital Secure Remote Payments (DSRP) enrolment are not required. No special flags are needed to identify Apple Pay transactions. If you have any questions, please reach out to your Apple Pay contact.

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

© 2018 Apple Inc. All rights reserved. Apple, the Apple logo, Apple Pay, and Wallet are all trademarks of Apple Inc., registered in the U.S. and other countries.

How Apple Pay Works

Presenting the Apple Pay Button

Within apps, **PassKit** provides the APIs that an app will use to determine if it is running on an Apple Pay capable device and if the device has been provisioned with payment cards that are supported. For the web, **Apple Pay JS** provides APIs that allow a website on Safari to check if the user has an Apple Pay capable device and if Apple Pay is set up.

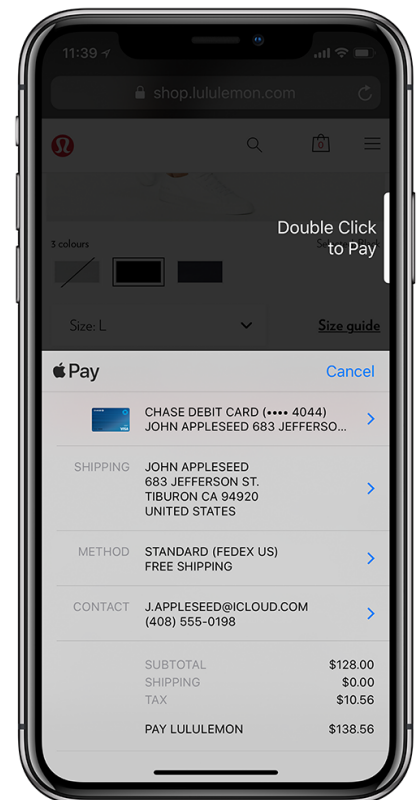
For Apple Pay button usage and design guidance, see [Apple Pay](#) and [Apple Pay on the Web](#) in the Human Interface Guidelines.

Presenting the Payment Sheet

When a user selects goods or services to buy or an amount to donate, and selects Apple Pay as the payment method, a merchant creates a payment request and asks PassKit (for apps) or Safari (for websites) to present the payment sheet to the user. The payment sheet must immediately follow the user tapping the Apple Pay button, without any interim screens or pop-ups except to prompt for necessary product details, such as size or quantity.

The app / website specifies the contents of the payment sheet, but it does not control the user's interaction with the sheet. The merchant must decide if it makes sense to present shipping and billing information, shipping method, and other line items to the user. The merchant should only request the information necessary to process and service the transaction.

For details on integrating the Apple Pay button and customizing the Apple Pay payment sheet for apps, see [Apple Pay](#) in the Human Interface Guidelines..



The Payment Token

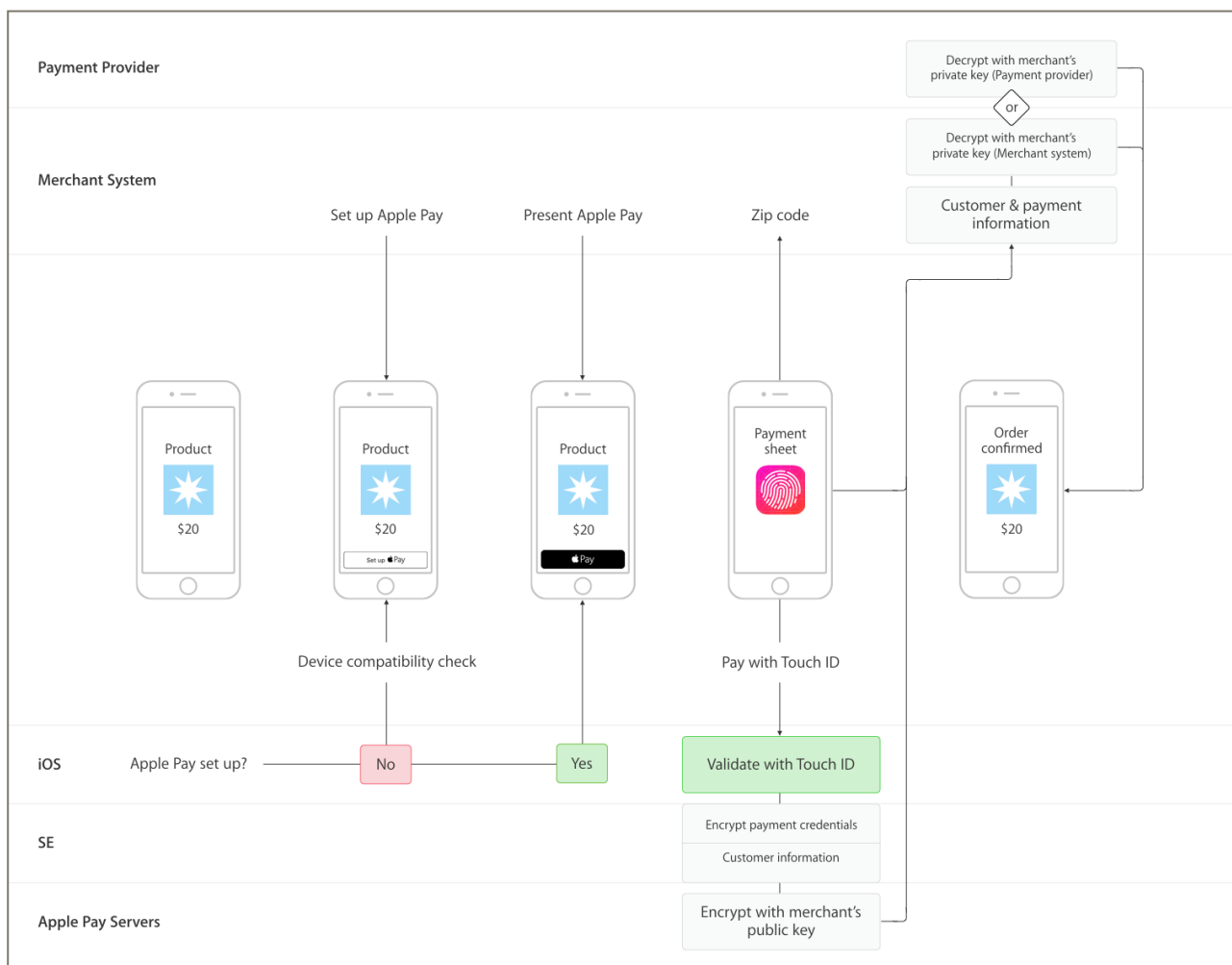
Once authorized by the user, the app or website receives a payment object, which contains an encrypted payment token from PassKit or Apple Pay JS. The payment token encapsulates the information needed to complete a payment transaction, including the device-specific account number (DPAN), the amount, and a unique, one-time-use

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

© 2018 Apple Inc. All rights reserved. Apple, the Apple logo, Apple Pay, and Wallet are all trademarks of Apple Inc., registered in the U.S. and other countries.

cryptogram. The encrypted payment token can be decrypted by the merchant with the certificate private key or by the payment processor on behalf of the merchant (recommended).

In some cases, a developer may not be set up as the merchant of record; in this case, the developer can decrypt the payment token and pass the decrypted token downstream to the appropriate merchant for processing with its respective processor. The app or website must make it clear that they are an intermediary party. For more details, see [Apple Pay](#) in the Human Interface Guidelines.



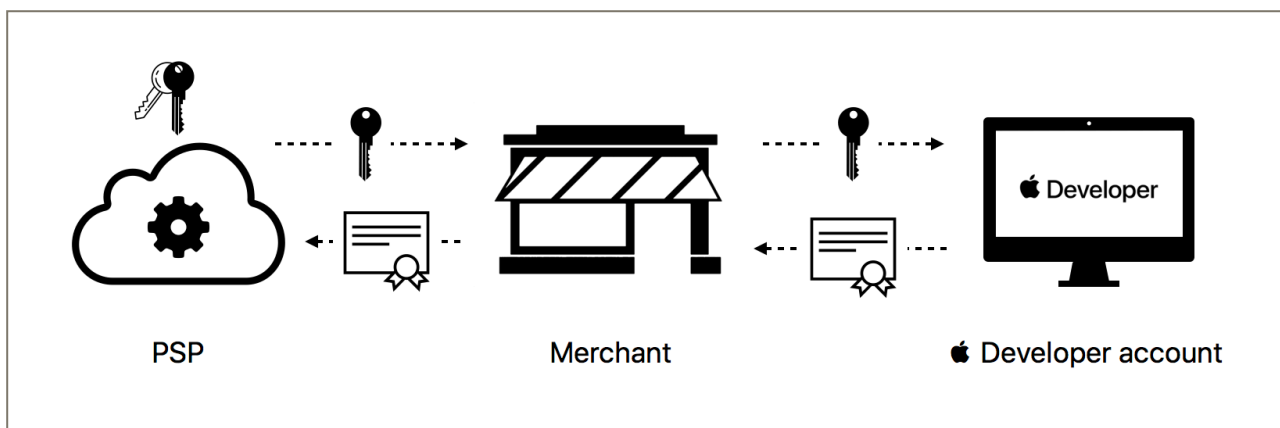
How Apple Pay works - high level flow

Merchant On-boarding

When setting up Apple Pay in an app or on the web, merchants must register a merchant identifier and set up cryptographic keys. Apple Pay uses encryption to ensure the secure and private transit of payment information. This payment information then needs to be decrypted before it can be processed. It is recommended that the payment processor manages this decryption, however it is also possible for the merchant to do this.

For full details of how a merchant sets up Apple Pay please see [here for app](#) and [here for web](#).

When the payment processor handles decryption, they need to generate the cryptographic keys and provide the public key via a Certificate Signing Request (CSR) to the merchant. The merchant will then upload this CSR to the Apple Developer portal. In return, Apple will provide the merchant a certificate which the payment processor will need to import.



Key generation and certificate sharing process

Payment processors will need to create a unique key pair for each merchant they on-board. It is recommended that payment processors allow merchants to manage the keys and certificates via the payment processor's merchant interface. A merchant should be able to generate the CSR and upload the returned certificate themselves within the merchant interface to streamline the setup process.

See [Appendix A1](#) for examples of how to set up Payment Processing Keys and Certificates.

Apple Pay on the Web

The above process is required regardless of whether the integration is taking place in app or on the web. When a merchant is using Apple Pay on the web, there are a few additional configuration steps required.

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

iOS apps contain digitally signed entitlements in the app binary that add another layer of security. These entitlements help validate and verify both the user and the merchant.

To provide a similar layer of security on the web, merchants will need to validate their URL and create a session via a server-side connection. Full information about setting up Apple Pay on the web can be found [here](#).

In addition to the payment processing certificate discussed in the previous section, merchants also need to create a session via a server-side interaction with Apple. This connection needs to be made via a 2-way TLS connection. It is not necessary for the payment processor to be part of this interaction, but it may be possible for the payment processor to manage this on behalf of the merchant if they are supporting Apple Pay on a hosted payment page or via a code library.

For reference, [Appendix A2](#) contains an example of how to setup a 2-way TLS keys and certificates for this stage of the integration.

A detailed flow of a full web transaction can be found in [Appendix A3](#)

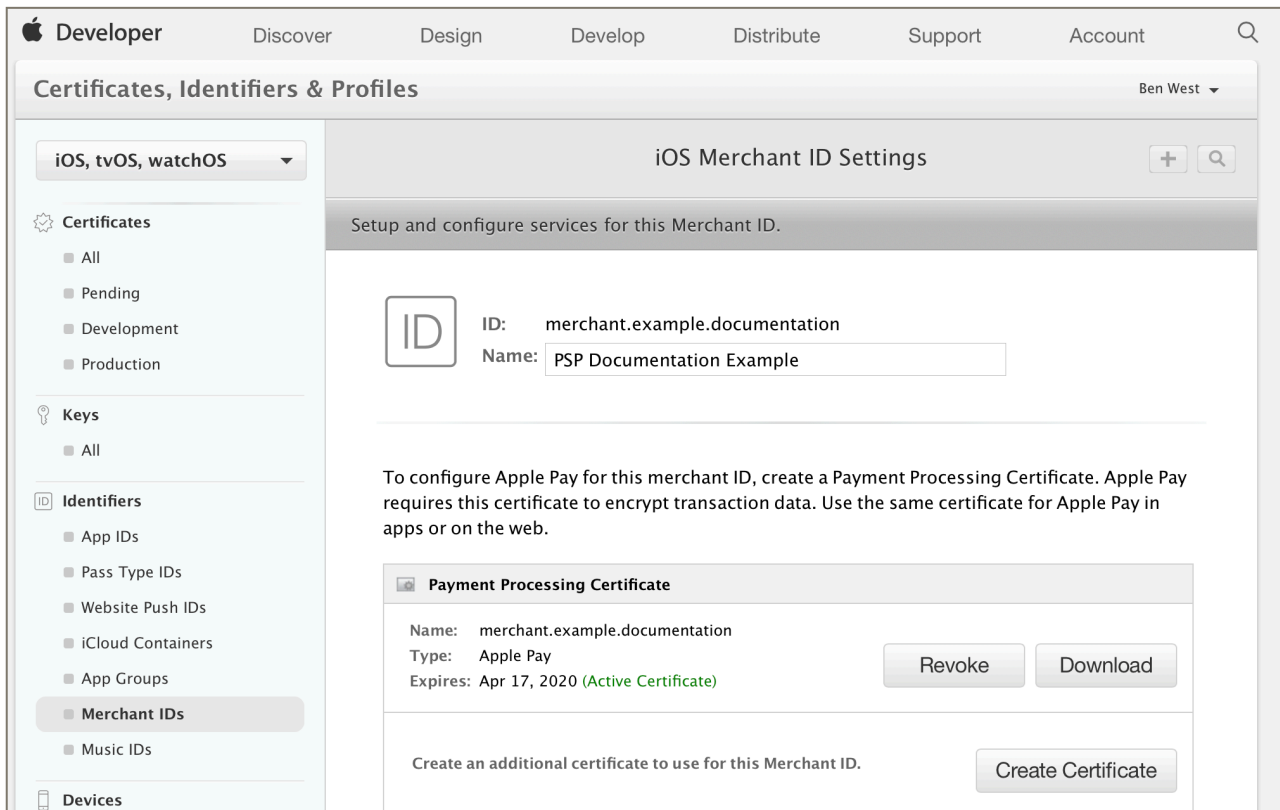
Key Management

Payment processors should make it easy for merchants to manage their Apple Pay keys and certificates. These certificates will expire over time, so it is recommended that payment processors provide mechanisms for merchants to roll keys and certificates without any downtime. Payment processors should allow merchants to have up to 2 active key pairs / certificates on their account at a time so that the merchant can transition from one key pair to a new set.

The key rolling process should follow the below steps (assuming existing keys are in place):

1. Generate new key pair and CSR in merchant interface
2. Upload new CSR to Apple Developer account & download new certificate
3. Upload new certificate to merchant interface
4. Activate the new keys/certificates
5. Revoke old keys/certificates

The below screenshot demonstrates how merchants can create new certificates in the Apple Developer account before they revoke the old certificate at the appropriate time.



Apple Pay configuration screen supporting multiple keys for key rolling

To simplify matters, it is also recommended that payment processors use the same keys / certificates across all their environments (sandbox and production). Apple Pay does not require separate keys per environment. If a payment processor requires separate keys per environment, it can add additional configuration complexity for the merchant and can delay setup and live enablement.

Best Practices

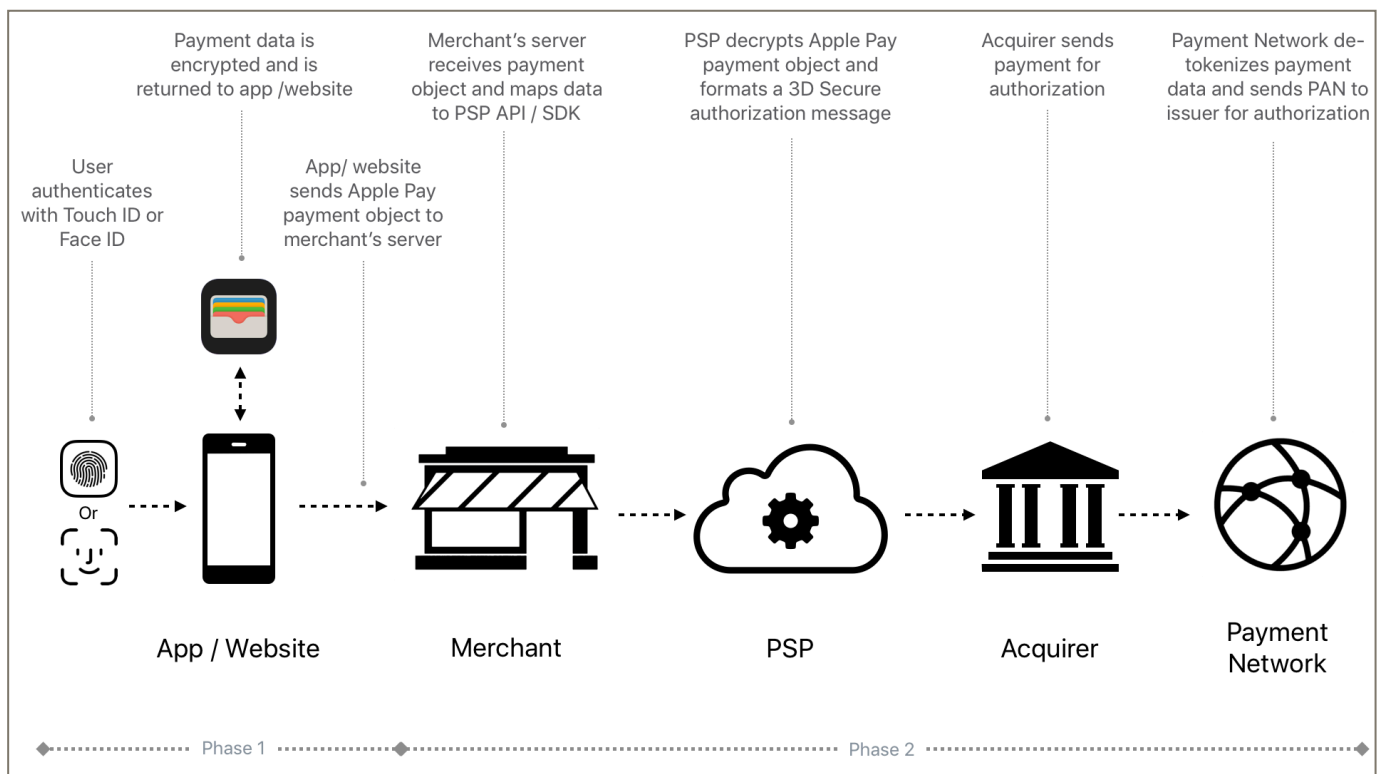
- Allow merchants to manage keys and certificates themselves via merchant interface
- Support multiple active key pairs to facilitate key-rolling without service disruption
- Use the same keys / certificates across sandbox and production environments to avoid complexity

Processing Apple Pay

The below diagram demonstrates the flow of an Apple Pay transaction, outlining the typical role the different entities play in the process. These roles may differ slightly depending on the way a payment processor presents their integration to the merchant (see [Payment Processor Integration Methods](#) section), however the interaction is always split into two stages:

Stage 1: *interacting with the Apple Pay APIs to generate the payment object*

Stage 2: *using the payment data to create a payment*



Typical Apple Pay flow showing roles in the process

Supported Transaction Types and Business Models

Apple Pay supports most eCommerce transaction types and offers the flexibility to accommodate a wide range of business models. The type of transaction flow a merchant implements will impact their user experience and key business metrics such as conversion rate and risk exposure.

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

© 2018 Apple Inc. All rights reserved. Apple, the Apple logo, Apple Pay, and Wallet are all trademarks of Apple Inc., registered in the U.S. and other countries.

Here are some examples of transaction types supported by Apple Pay:

Transaction type	Description	Example
Authorization & capture	Reserve funds on a customer's account and transfer money to your bank account	Online shopping where merchandise is readily available for shipping
Authorization & delayed capture	Reserve funds on a customer's account and transfer money to your bank later	Pre-order, delivery service with tip, order online pick up in store
Authorization & capture with different amount	Reserve funds on a customer's account and transfer money to your bank account once an order is successfully completed for an amount higher or lower than authorized	Groceries delivery service where some items are priced by actual weight
Authorization unknown amount & capture	Authorize a transaction with unknown amount, and charge the amount for the service later	Taxi ride
Partial shipment	Divide a purchase into multiple payments for goods that are not shipped together*	Multi-item purchase, with more than one deliveries
Fixed Subscription	Handle repeating payments at a regular frequency and with a fixed amount*	Gym monthly membership
Flexible Frequency Subscription	Handle payments for services where frequency is inconsistent or user has choice to reduce frequency*	Meal subscriptions, where user can skip deliveries, reschedule delivery dates, change frequency of deliveries
Flexible Amount Subscription	Handle payments for services where amount varies base on your consumption*	Utility bill, subscription with a promotion on the first month
Managed Subscription	Handle payments for services where the user can vary frequency and amount of services received*	Meal delivery service with high flexibility
Canceled transactions	Reverse money for canceled, reimbursed or disputed transactions	Chargeback, voided transaction, refund

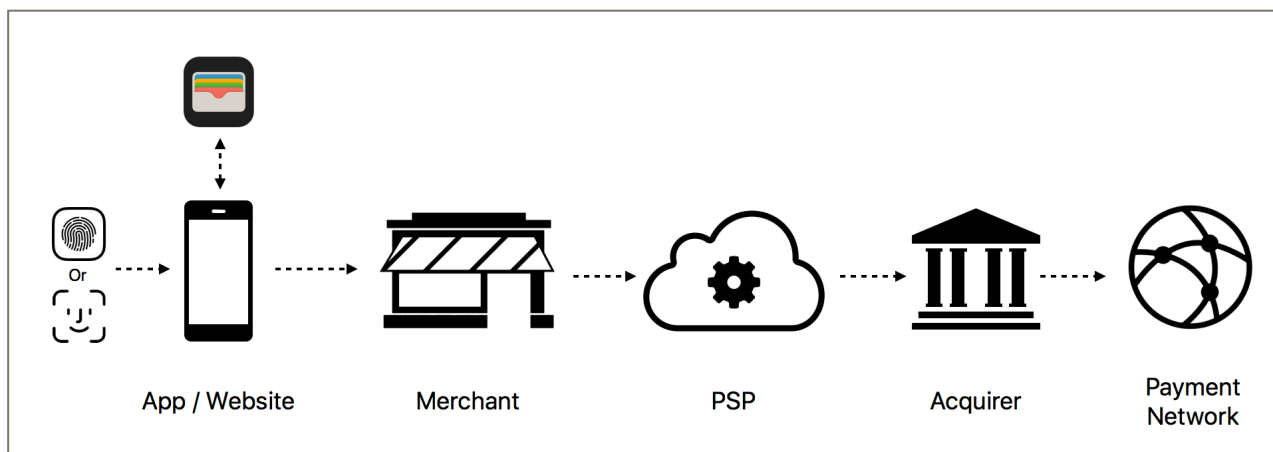
* Transaction type may not be available in all countries/regions.

Recurring Payments and Follow-on Transactions

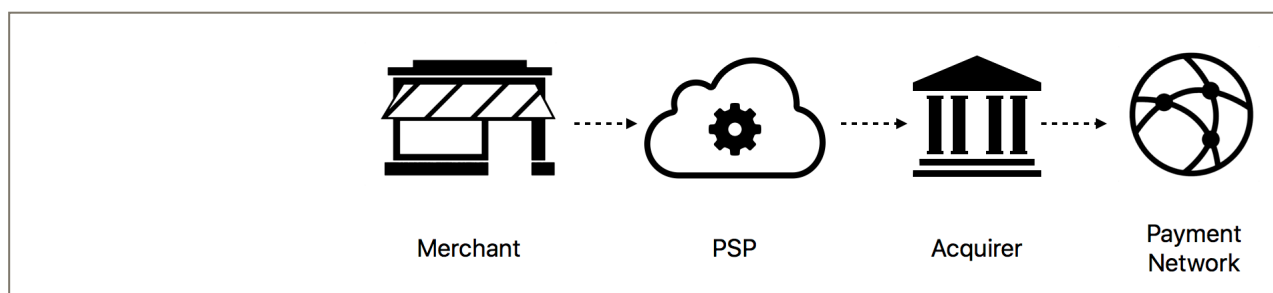
As well as customer-initiated transactions, Apple Pay can also be used for making recurring payments or other follow-on transactions where the merchant initiates the transaction.

After the consumer creates the initial transaction, the merchant or payment processor can store some elements of the payment to create a follow-on transaction at a later date.

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.



Initial transaction (initiated by the user in the app / on the web)



Follow-on transaction (initiated by the merchant from their server)

Depending on the payment network, subsequent transactions may require flagging in a different way to indicate that the transaction is being initiated by the merchant. As an example, Visa's *Merchant Initiated Transactions (MIT)* specification, and MasterCard's *Digital Secure Remote Payment (DSRP)* specification outline how to format authorization messages for these types of payment. If a payment processor is interested in supporting these types of transaction they are best to enquire with their acquiring partner or the payment networks for further information.

Apple Pay Payment Object

Once the user authenticates with Touch ID or Face ID, the Apple Pay APIs will call back the app / website with a payment object containing the payment data (encrypted) as well as customer information.

Below is an example of the full Apple Pay payment object structure a merchant's app / website can receive when asking for all the customer information from the payment sheet.

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

```

{
  token:{
    paymentData: {
      version:"EC v1",
      data:"3+f4oOTwPa6f1UZ6tG...CE=",
      signature:"MIAGCSqGS Ib3DQ...AAAA==",
      header:{
        ephemeralPublicKey:"MFkwEK...Md==",
        publicKeyHash:"10CnXdMv...D1I=",
        transactionId:"32b...4f3"
      }
    },
    paymentMethod:{
      displayName:"Visa 1234",
      network:"Visa",
      type:"debit"
    },
    transactionIdentifier:"32b...4f3"
  },
  billingContact:{
    addressLines:[
      "1 Street",
      ""
    ],
    administrativeArea:"",
    country:"United Kingdom",
    countryCode:"GB",
    familyName:"Appleseed",
    givenName:"John",
    locality:"London",
    postalCode:"AB12 3CD",
    subAdministrativeArea:"",
    subLocality:""
  },
  shippingContact:{
    addressLines:[
      "1 Street",
      ""
    ],
    administrativeArea:"",
    country:"United Kingdom",
    countryCode:"GB",
    familyName:"Appleseed",
    givenName:"John",
    locality:"London",
    postalCode:"AB12 3CD",
    subAdministrativeArea:"",
    subLocality:"",
    phoneNumber:"01234 567890",
    emailAddress:"john.appleseed@apple.com"
  }
}

```

Apple Pay payment object in JSON format

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

© 2018 Apple Inc. All rights reserved. Apple, the Apple logo, Apple Pay, and Wallet are all trademarks of Apple Inc., registered in the U.S. and other countries.

The main elements of the above payment object are explained as follows:

Element	Description
<code>token.paymentData</code>	The encrypted payment credentials and associated cryptography information
<code>token.paymentMethod</code>	Ancillary information about the cardholder's card which was provisioned to the device
<code>billingContact</code>	[<i>optional</i>] The billing address information for the user (only provided if the merchant requested this information from the payment sheet)
<code>shippingContact</code>	[<i>optional</i>] The shipping address and contact information for the user (only provided if the merchant requested this information from the payment sheet)

Learn more about the structure of an Apple Pay payment token [here](#).

Authorizing the Payment

Once the user has authorized themselves on the payment sheet the app / website will receive the Apple Pay payment object outlined in the previous section.

At this point the user has verified their identity using Touch ID or Face ID but a payment has not yet taken place.

The merchant needs to pass the token information to the payment processor who can then create an authorization via the payment networks. As stated previously, the merchant can decrypt the payment data themselves, but more commonly the payment processor will manage this complexity on the merchant's behalf.

Decrypting the Payment Data

The Secure Element encrypts the token's payment data using either elliptic curve cryptography (ECC) or RSA encryption. The encryption algorithm is selected by the Secure Element based on the payment request. Most regions use ECC encryption. RSA is used only in regions where ECC encryption is unavailable due to regulatory concerns.

Unless advised otherwise, payment processors should focus on ECC encryption / decryption.

Full instructions outlining the process for decrypting the payment data can be found [here](#).

Java sample code for managing the decryption can be downloaded [here](#)

Constructing the Authorization Message

Once the payment data has been decrypted, it needs to be sent for authorization. Apple Pay has been designed to use the same data elements as a 3D Secure transaction. Creating a successful authorization should be a case of mapping the Apple Pay fields to existing payment fields used for card payments.

Once decrypted, the following fields from the Apple Pay data should be mapped to the data fields used for 3D Secure compatible card payments in the following table:

Apple Pay Field	Representative Card Field
<code>data.applicationPrimaryAccountNumber</code>	PAN
<code>data.applicationExpirationDate (YYMMDD)</code>	Expiration Date
<code>data.paymentData.onlinePaymentCryptogram</code>	CAVV / UCAF
<code>data.paymentData.eciIndicator</code>	ECI

American Express

Payments using an American Express card may require slightly different mapping to that outlined above. In addition to the above mapping, please use the following table for guidance on mapping Amex payment fields:

Apple Pay Field	Amex Card Field
<code>data.deviceManufacturerIdentifier</code>	Token Requestor ID
<code>data.paymentData.onlinePaymentCryptogram</code>	Token Block A
<code>[all zeros] fill field value with zeros</code>	Token Block B

NOTE: above tables only for guidance, always check with your acquiring/payment network partner for correct mapping

Best Practices

- Manage the decryption of the payment data to simplify merchant integrations
- Focus on ECC decryption unless advised otherwise
- Support decrypted Apple Pay data as well as encrypted Apple Pay data
- Support all transaction types including recurring and partial shipment payments
- Consume all Apple Pay data, including billing and shipping contact information, not just payment data

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

Payment Processor Integration Methods

Payment processors will usually offer merchants a range of ways to consume their services. There may be various integration types to suit different complexity merchants, ranging from simple hosted payment forms all the way through to direct API (server-to-server) integrations.

In order for merchants to get the most from Apple Pay, the Apple Pay button is best placed as early as possible in the checkout experience, typically on the product or basket summary page.

For this reason, it is best for a payment processor to make Apple Pay available in such a way that the merchant can control the location of the Apple Pay experience in their checkout flow.

Apple strongly recommends payment processors to build their Apple Pay support via a direct API offering, or by including Apple Pay in JavaScript or iOS code libraries.

Direct API Integration

Payment processors should always support this method of integration for Apple Pay as it give merchants complete control over how they integrate Apple Pay into their website or app.

In this scenario the merchant is responsible for the interactions with the Apple Pay APIs. Once the merchant completes this stage, the payment processor needs to consume the payment data.

It is likely the payment processor already offers API fields for customer address and contact information, so the payment processor should only need to extend their API to support the encrypted payment data.

JavaScript & iOS Code Libraries

Payment processors typically offer these products to simplify and speed up integrations, and the same can apply to Apple Pay. Payment processors may wish wrap around the Apple Pay APIs to reduce the amount of effort a merchant needs to adopt Apple Pay as a payment option.

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.© 2018 Apple Inc. All rights reserved. Apple, the Apple logo, Apple Pay, and Wallet are all trademarks of Apple Inc., registered in the U.S. and other countries.

It is recommended that the JavaScript and iOS libraries should be built to simplify the integration of Apple Pay, but still give the merchant control of where to present the Apple Pay button in their checkout flow.

Hosted Payment Forms

Apple Pay is designed to accelerate the checkout process. Whilst it is possible to support Apple Pay on a hosted payment form, its potential will not be maximised. By the time the user has reached the hosted payment form they have likely been asked for their address and contact information – steps which could have been bypassed if Apple Pay had been presented earlier.

If payment processors are interested in supporting Apple Pay on a hosted payment form they should discuss this with their Apple Pay contact to evaluate the best approach.

iframes

Apple Pay on the web uses a number of measures to ensure transactions remain secure and private. One of these measures is to validate the domain which will be calling the Apple Pay APIs. For this reason, any attempt to call the APIs from within an iframe will not be successful as it will not match the validated domain of the parent page.

Best Practices

- Always support Apple Pay via a direct API integration
- Consider including Apple Pay via JavaScript and iOS code libraries to simplify integration
- Ensure code library integrations empower merchants to place the Apple Pay experience where they want

Sandbox Testing

Merchants and payment processors can perform sandbox testing for Apple Pay using a range of test cards. To provision these cards to the Apple Pay wallet, the user will need to sign into iCloud on their test device using a Sandbox Tester account.

Further information can be found [here](#).

When a test card is added to Apple Pay it will be provisioned using a test DPAN which will be used in Apple Pay transactions. It is recommended that payment processors support these test DPANs as acceptable values in their own sandbox environments so that successful transactions can be simulated. A list of test DPANs are available from your Apple Pay contact for the test cards found [here](#).

Best Practices

- Support Apple Pay test DPANs on payment processor's sandbox environment.

Documentation

Payment processors should provide detailed technical documentation to their merchants for all the ways they support Apple Pay via their platform. It is recommended that payment processors provide links to any existing content on the [Apple Pay developer website](#), instead of recreating similar content on their own site.

Where possible, it is recommended that payment processors use code examples and step-by-step instructions to clearly explain the requirements for integrating Apple Pay.

Reporting

It is recommended that payment processors identify Apple Pay transactions in all transaction information and reporting made available to merchants.

Appendices

Appendix A1

Generating a Payment Processing Certificate

1. The party responsible for decryption then generates a Certificate Signing Request (CSR) for a Payment Processing Certificate per the following (and keeps the private key safely):

- a) Command-line tools

- (i) OpenSSL

- Generate key pair in a key file

```
openssl ecparam -genkey -name prime256v1 -out ecckey.key
```

- Generate CSR from key pair in key file

```
openssl req -new -sha256 -key ecckey.key -out ecccertreq.csr -subj /  
CN=www.mydomain.com
```

- (ii) Keytool

- Generate key pair in a pkcs12 file

```
keytool -genkeypair -keyalg EC -keystore ecckeystore.p12 -storetype  
pkcs12 -keysize 256 -alias ecckeyname -dname CN=www.mydomain.com
```

- Generate CSR from key pair in pkcs12 file

```
keytool -certreq -sigalg SHA256withECDSA -alias ecckeyname -file  
ecccertreq.csr -keystore ecckeystore.p12 -storetype pkcs12
```

- b) Keychain Access (Mac only)

- (i) In the Applications folder on your Mac, open the Utilities folder and launch Keychain Access
- (ii) Within the Keychain Access drop down menu, select *Keychain Access > Certificate Assistant > Request a Certificate from a Certificate Authority*
- (iii) In the Certificate Information window, enter the following information:
 - In the User Email Address field, enter your email address

- In the Common Name field, create a name for your private key (e.g., John Doe Dev Key)
- The CA Email Address field should be left empty
- In the "Request is" group, select the "Saved to disk" option
- Select "Let me specify key pair information"

(iv) Click Continue within Keychain Access and select the file location

(v) Set the Key Pair Information to the following:

- Algorithm: ECC
- Key size: 256 bits

(vi) Click Continue within Keychain Access to complete the CSR generating process

2. Merchant then uploads the Payment Processing Certificate CSR as shown [here](#)

3. Download (and backup) the Apple signed Payment Processing Certificate, *apple_pay.cer*

4. The party responsible for decryption then installs the Payment Processing Certificate generated as aforementioned to prepare for decrypting transactions

a) Command-line tools

(i) OpenSSL

- convert *apple_pay.cer* to PEM

```
openssl x509 -inform DER -in apple_pay.cer -out apple_pay.pem
```

- Import merchant certificate and private key to generate P12

```
openssl pkcs12 -export -out ecckeystore.p12 -inkey ecckey.key -in apple_pay.pem
```

(ii) Keytool (ensure relevant [Apple Root and Intermediate certificates](#) are installed in your truststore)

```
keytool -import -keystore ecckeystore.p12 -storetype pkcs12 -alias ecckeyname -trustcacerts -file apple_pay.cer
```

b) Keychain Access (Mac only)

- a) Double click the *apple_pay.cer* file to install in Keychain Access

Appendix A2

Generating a Merchant Identity Certificate

1. Merchant generates a Certificate Signing Request (CSR) for a Merchant Identity Certificate per the following (and keeps the private key safely):

- a) Command-line tools

- (i) OpenSSL

- Generate key pair in a key file

```
openssl req -new -newkey rsa:2048 -nodes -out rsacertreq.csr -keyout  
rsakey.key -subj /CN=www.mydomain.com
```

- (ii) Keytool

- Generate key pair in a pkcs12 file

```
keytool -genkeypair -keyalg RSA -keystore rsakeystore.p12 -storetype  
pkcs12 -keysize 2048 -alias rsakeyname -dname CN=www.mydomain.com
```

- Generate CSR from key pair in pkcs12 file

```
keytool -certreq -alias rsakeyname -file rsacertreq.csr -keystore  
rsakeystore.p12 -storetype pkcs12
```

- b) Keychain Access (Mac only)

- (i) In the Applications folder on your Mac, open the Utilities folder and launch Keychain Access

- (ii) Within the Keychain Access drop down menu, select *Keychain Access > Certificate Assistant > Request a Certificate from a Certificate Authority*

- (iii) In the Certificate Information window, enter the following information:

- In the User Email Address field, enter your email address
- In the Common Name field, create a name for your private key (e.g., John Doe Dev Key)
- The CA Email Address field should be left empty
- In the "Request is" group, select the "Saved to disk" option

- (iv) Click Continue within Keychain Access to complete the CSR generating process

2. Merchant then uploads the Merchant Identity Certificate CSR

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

3. Download (and backup) the Apple signed Merchant Identity Certificate, *merchant_id.cer*
4. The merchant environment then installs the *merchant_id.cer* for use in securing the 2-way TLS handshake
 - a) Command-line tools
 - (i) OpenSSL
 - convert *merchant_id.cer* to PEM

```
openssl x509 -inform DER -in merchant_id.cer -out merchant_id.pem
```

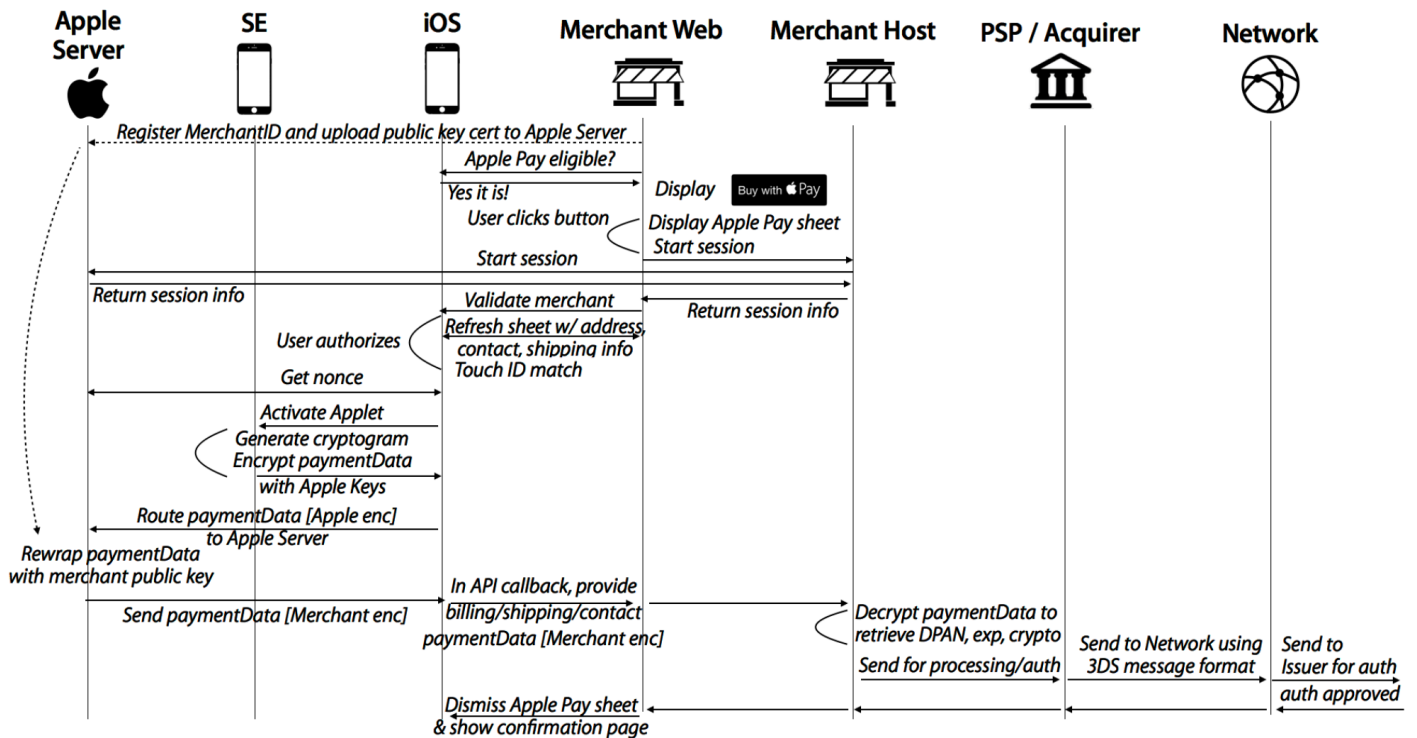
 - Import merchant certificate and private key to generate P12

```
openssl pkcs12 -export -out Certificates.p12 -inkey rsakey.key -in merchant_id.pem
```
 - (ii) Keytool (ensure relevant Apple Root and Intermediate certificates are installed in your truststore)

```
keytool -import -keystore rsakeystore.p12 -storetype pkcs12 -alias rsakeyname -trustcacerts -file merchant_id.cer
```
 - b) Keychain Access (Mac only)
 - a) Double click the *merchant_id.cer* file to install in Keychain Access
5. Merchant domain validation: <https://developer.apple.com/documentation/applepayjs#2193397>

Appendix A3

Web payment flow



Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

© 2018 Apple Inc. All rights reserved. Apple, the Apple logo, Apple Pay, and Wallet are all trademarks of Apple Inc., registered in the U.S. and other countries.

Change Log

Date	Version Number	Notes
28 March 2018	1.0	First Version
28 August 2018	1.1	Updated logo. Updated links
30 April 2019	1.2	Fixed links
13 June 2019	1.3	Fixed links

Apple makes no representations or warranties (whether express or implied, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of the use of or reliance on this document), and expressly disclaims all liability, with regard to the subject matter contained herein.

© 2018 Apple Inc. All rights reserved. Apple, the Apple logo, Apple Pay, and Wallet are all trademarks of Apple Inc., registered in the U.S. and other countries.